

Project Build Log

Swarm-Dynamic Federated Adversarial Defense for Medical IoT (MIoT)

A complete, step-by-step record of everything built in this repository: what was installed and why, which module was written at each stage, what every module does, what was measured, and what remains. Written to be answerable end-to-end in a review.

Repository Federated-Adversarial-Defenses-for-Medical-IOT

Code 48 Python files · 6,720 lines · 41 library modules + 7 entry-point scripts

Runtime Python 3.14.6 (arm64 / Apple silicon), virtualenv at `.venv/`

Pipeline digital twin → GNN detector → GAN adversary → DP-SGD → Byzantine-robust aggregation

Progress Steps 1–6 built and executed · Steps 7–10 not yet implemented

Document generated 19 August 2026

Contents

§1	The project in one page — what it is and what it claims
§2	Step 1 — Environment: what was installed, and why each package
§3	Step 2 — The MQTT digital twin (5 devices, 6 attacks, 6 benign confounders)
§4	Step 3 — Dataset construction: 30-feature schema, splits, non-IID sharding
§5	Step 4 — Detector architectures: MLP vs Transformer vs GNN
§6	Step 5 — Federated learning: FedAvg, own simulator + real Flower
§7	Step 6 — GAN adversarial engine and adversarial training
§8	Complete module reference — every file, what it does
§9	All measured results, in one place
§10	Design decisions, and how to defend each one
§11	Anticipated panel questions with answers
§12	Honest status: what is not done, and known issues
§13	How to reproduce every result — exact commands

READING THIS IN A HURRY

If you only have ten minutes before the review, read §1 (the claim), §9 (the numbers), §11 (the questions), and §12 (what you must not overclaim).

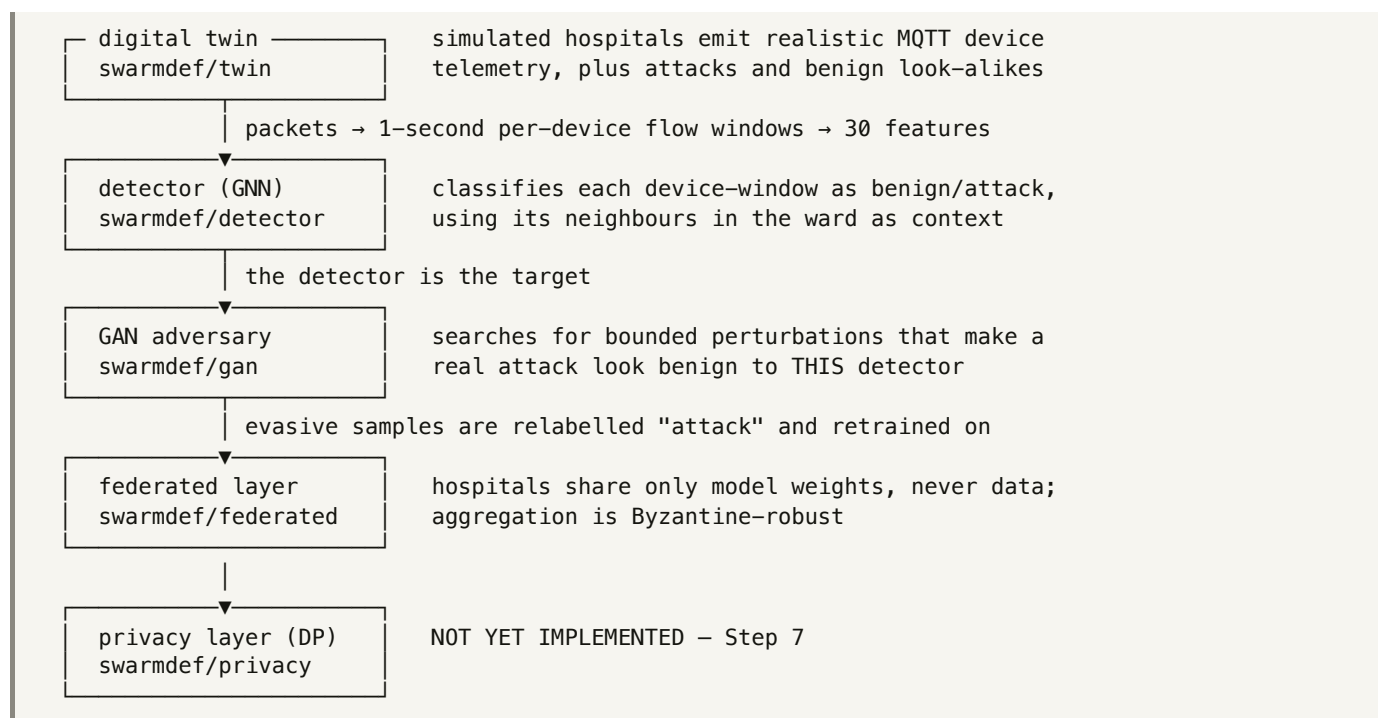
1. The project in one page

The problem

Hospital networks now carry hundreds of connected medical devices — infusion pumps, ventilators, bedside monitors — on the same segment. These devices are attractive targets and poorly defended, but the traffic that would let you train an intrusion detector is patient-linked clinical data, so it is legally and ethically impossible to pool it across hospitals. A hospital training alone sees only the narrow slice of the threat landscape that happens to hit it.

The system built here

Five layers, wired into one closed loop:



The security claim, stated precisely

A blind spot discovered by **one** hospital's adversary is patched for **all** hospitals after the next aggregation round — without any hospital sharing a single packet of patient traffic.

Step 6 is where this becomes measurable, and it does hold in the measured runs: an undefended federated detector loses **27.8 percentage points of attack recall** to the GAN (99.2% → 71.3%, evasion 28.3%), while the adversarially-trained federated detector loses only **3.3 points** (99.2% → 95.8%, evasion 4.3%) — and it is *also* slightly better on clean traffic.

Chronology at a glance

Step	What was built	Status	Evidence on disk
1	Environment, dependency install, PyG smoke test, Mosquitto broker	done	.venv/, requirements.txt
2	MQTT digital twin: devices, attacks, benign events, broker, live monitor	done	swarmdef/twin/ (9 modules)
3	30-feature schema, feature extractor, scaler, splits, non-IID shards, CIC loader, 6-check verifier	done	data/processed/, data/hospitals/
4	MLP / Transformer / GraphSAGE / GAT detectors, centralised baseline, binary + 7-way	done	step4_baseline_*.json, 4 .pt models
5	Federated simulator, HospitalClient, 5 aggregators, real Flower deployment	done, one caveat	step5_fedavg_gnn.csv
6	Perturbation GAN, adversarial engine, adversarial training, budget sweep	done	step6_adversarial.json
7	Differential privacy (Opacus DP-SGD + accountant)	not built	config + hooks only
8	Byzantine attack experiments (aggregator comparison under poisoning)	code done, no run	aggregators implemented
9	Final figure set / full evaluation sweep	partial	11 figures generated
10	Report writing / packaging	not started	—

2. Step 1 — Environment

Everything runs inside a project-local virtual environment at `.venv/` on **Python 3.14.6**, arm64. Nothing is installed system-wide.

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
```

2.1 What was installed, and what each package does here

Package	Version	Role in this project
<code>torch</code>	2.13.0	All neural networks: the three detector architectures, the GAN generator, every training loop.
<code>torch-geometric</code>	2.8.0	Graph neural network layers (<code>SAGEConv</code> , <code>GATConv</code> , <code>GCNConv</code>) and the <code>Data / Batch</code> graph containers. Pure-Python build — <code>torch-scatter</code> is deliberately <i>not</i> required, which is what usually breaks PyG installs.
<code>flwr</code> (Flower)	1.33.0	The real federated-learning framework: <code>ClientApp</code> / <code>ServerApp</code> , parameter serialisation, the <code>FedAvg</code> strategy that we subclass.
<code>ray</code>	2.55.1	Pulled in by Flower; it is the backend that runs each simulated hospital in its own worker process.
<code>opacus</code>	1.6.0	Differential privacy — DP-SGD gradient clipping, Gaussian noise, RDP accountant. Installed and ready, but Step 7 is not yet written.
<code>paho-mqtt</code>	2.1.0	MQTT client library. Publishes twin telemetry to the broker and subscribes in the live monitor.
<code>pandas</code>	3.0.5	All tabular data handling: flow-window frames, hospital shards, CSV persistence.
<code>numpy</code>	2.5.2	Feature maths, aggregation rules (all aggregators operate on plain NumPy arrays), metrics.
<code>scikit-learn</code>	1.9.0	Used only for the logistic-regression <i>linear probe</i> in the dataset verifier — a deliberately weak model used to prove the features carry signal.
<code>scipy</code>	1.18.0	Numeric support.
<code>matplotlib</code>	3.11.1	Every result figure.
<code>pyyaml</code>	6.0.3	Loads <code>configs/default.yaml</code> into the typed <code>Config</code> object.
<code>tqdm</code> , <code>tabulate</code>	4.70 / 0.10	Progress bars and console tables.
<code>mosquitto</code>	Homebrew	The MQTT broker itself — a native binary, not a Python package.

2.2 Two decisions taken in Step 1 that are worth defending

PyTorch Geometric was installed and smoke-tested in Week 1, not Week 5

The original plan scheduled PyG for the GNN phase. PyG is the single most common source of environment failure in this kind of project (CUDA/compiler mismatches, `torch-scatter` build errors). Installing it first means that if the environment was going to fail, it failed on day one when there was time to fix it — rather than in the middle of the modelling phase. The install uses PyG's pure-Python path, so no compilation is needed.

Mosquitto is installed natively (Homebrew), not via Docker

The original spec called for a Dockerised broker. Docker Desktop is a multi-gigabyte dependency and a background daemon for what is, in practice, a single ~400 KB binary. `swarmdef/twin/broker.py` detects and starts the native binary automatically, and also detects an *already running* broker (including a container) and simply uses it. So the Docker path still works — `docker run -d -p 1883:1883 eclipse-mosquitto` — it is just not required.

2.3 One deliberate non-default: CPU, not MPS

`swarmdef/utils/seed.py::resolve_device()` resolves `auto` to CUDA if present, otherwise **CPU** — it will not silently select Apple's MPS backend. Reason: Opacus' per-sample gradient hooks and several PyG scatter kernels either error or silently fall back on MPS, which would corrupt Step 7's privacy accounting. MPS can still be requested explicitly. The models are small enough (6.5k–69k parameters) that CPU training takes 1–16 seconds per architecture.

3. Step 2 — The MQTT digital twin

There is no public dataset of hospital MQTT traffic with labelled attacks against infusion pumps. So the first substantial build was a simulator that produces one: `swarmdef/twin/`, nine modules, ~1,750 lines.

3.1 The core abstraction: `Packet`

`twin/packet.py` defines a single record type for one observable network event: timestamp, source, destination, MQTT topic, control-packet type (PUBLISH / SUBSCRIBE / CONNECT / PINGREQ / DISCONNECT), protocol, payload and header lengths, QoS, retain, dup, TCP flags, the decoded clinical payload, a `vital_z` implausibility score, and the ground-truth label.

This one record is why the twin can run in two modes with *identical* semantics: **online**, where packets are genuinely published through Mosquitto, and **offline**, where they are generated in-process for the dataset builder. Same physics, same attacks, same labels — so the offline dataset is a faithful stand-in for the live demo, and this is defensible when challenged.

`Packet.payload_entropy()` computes real Shannon entropy over the serialised JSON payload (structured telemetry lands around 4–5 bits/byte; encrypted or randomised blobs approach 8). `FlowWindow` groups packets for one device over one second and applies the labelling rule: **a window is malicious if it contains any attack packet**.

3.2 Five device types, driven by real stochastic processes

`twin/devices.py`. Each vital sign is a bounded **Ornstein–Uhlenbeck process**: mean-reverting drift toward a baseline plus Gaussian noise, then clamped to clinical limits. Values therefore drift smoothly and stay physiologically plausible, which matters — if vitals were pure noise, "detect the anomaly" would be trivial thresholding.

Device	Payload	Signals modelled
PatientMonitor	140 B	heart rate, SpO2, respiration rate, systolic BP, temperature
InfusionPump	110 B	flow rate mL/h, reservoir %, line pressure, battery % (QoS 1 — dosing commands are at-least-once)
Ventilator	160 B	tidal volume, PEEP, FiO2, airway pressure (QoS 1)
WearableSensor	70 B	heart rate, step count, skin temperature
EnvironmentSensor	80 B	room temperature, humidity, CO2 ppm
Gateway	300 B	not a clinical device — the ward's observation point

WHY THE GATEWAY NODE EXISTS — A REAL BUG THAT WAS FOUND AND FIXED

Reconnaissance and Spoofing address no specific device: a subnet scan touches many hosts once each. Without a gateway node, every one of those packets had nowhere to be attributed and was silently dropped — **two entire attack classes were vanishing from the dataset**. The Gateway is the IDS span port: it absorbs unattributed segment traffic *and* carries its own realistic benign baseline (EHR sync 38%, PACS imaging 17%, DNS 18%, NTP 12%, broker keepalives 15%). That baseline is essential — without it every packet on the gateway would be hostile and the node would be trivially separable.

Each device publishes to `hospital/<hospital_id>/<device_type>/<device_id>/telemetry`. `build_fleet()` guarantees a patient monitor and an infusion pump (the primary attack targets) and samples the rest, so hospitals genuinely differ in device population — non-IID at the *source*, not just in the labels.

3.3 Six attack scenarios — each with a genuinely different signature

`twin/attacks.py`. Every scenario is a finite-duration state machine with two hooks: `mutate(pkt)` tampers with a packet the fleet was going to send anyway, and `inject(t, dt)` emits *additional* hostile packets. That split is the whole point: a MITM only rewrites existing traffic (volume unchanged — invisible to any rate-based IDS), while a DDoS adds thousands of packets (volume explodes, but each payload is well-formed). A detector looking at one feature family will always miss one of the two.

Attack	Mechanism and detectable signature
DDoS	Volumetric flood up to 400 pkt/s, ramping over the first 20% of the window. Packet rate and byte rate spike, inter-arrival collapses, SYN ratio climbs to ~70%, sources fan out across a <i>bounded</i> botnet pool.
MITM	Rewrites a clinical value in transit — suppress an alarm or inflate an infusion rate. Volume unchanged. Tells: clinical implausibility (<code>vital_zscore_max</code>) and a ~12% DUP-flag artefact from the interception proxy. The victim device then alarms, exactly as it would in a genuine emergency.
FirmwareTamper	Forged OTA firmware push: a slow burst of 8–32 KB chunks, entropy 7.6–8.0 bits/byte, on an OTA topic the device never normally sees, with <code>retain=true</code> so the payload sticks for future subscribers.
Spoofing	A rogue host on the same subnet publishes as a legitimate device. Duplicate identity from a new source address, CONNECT storm as the impostor fights the real client, and telemetry that guesses a plausible constant instead of continuing the true physiological walk.
Recon	Topic enumeration and port scanning. Low volume, but very high topic entropy and destination fan-out — the opposite shape of a flood. RST ratio climbs as closed ports refuse.
Mirai	Two phases: a CONNECT brute-force burst (~45 attempts/s) for the first 35% of the window, then a machine-regular 0.75 s C2 beacon with small high-entropy UDP payloads. The <i>regularity</i> is itself the anomaly.

3.4 Six benign confounder events — the module that fixed a broken benchmark

THIS IS THE SINGLE MOST DEFENSIBLE PIECE OF ENGINEERING IN THE PROJECT

The first version of the dataset was solved by **plain logistic regression at 99.9% accuracy**. That is a defective benchmark: with the detector already at ceiling there is no headroom for the GAN to degrade, no visible cost to DP noise, and no measurable damage from a poisoned client — every downstream plot would be a flat line at 1.0. The fix was **not** to add artificial noise; it was three corrections toward realism.

`twin/events.py` adds legitimate hospital activity that *structurally resembles* an attack but is labelled BENIGN:

Benign event	Confounds	What it does
CodeBlue	MITM	Cardiac arrest: vitals swing hard and legitimately. Plus a ward-wide response — the device alarms, staff chart against the EHR at the gateway, neighbouring devices are handled and report more.
AuthorisedFirmwareUpdate	FirmwareTamper	Signed rollout from the biomed management server. Byte-for-byte identical shape; only provenance differs.
ImagingTransfer	exfiltration	Radiology study moving to PACS: sustained high byte rate, 1200–1460 B packets, entropy 7.2–7.9.
DeviceReboot	Mirai brute force / SYN flood	Power cycle: CONNECT storm at ~25/s, then normal service.
ShiftHandover	volumetric spike	Ward-wide sync: every device reconnects and backfills.
NetworkCongestion	MITM DUP artefact	Switch congestion sets DUP and RST flags across the whole fleet.

THE EMPIRICAL JUSTIFICATION FOR USING A GNN — MEMORISE THIS ONE

CodeBlue and MITM are deliberately identical in the victim device's own feature vector. The MITM code comments say so explicitly: the tampered value uses the same direction convention and the same sigma-scaled magnitude range as CodeBlue, so `vital_zscore_max` alone *cannot* separate them.

What separates them is the ward's reaction. A real resuscitation is a correlated, ward-wide event — rapid-response paging, EHR charting at the gateway, neighbouring devices handled. An MITM produces **one alarming device surrounded by a completely calm ward**.

That evidence does not exist in the victim's own 30-D vector. No per-window model can make this call. A graph model can, because it sees whether the neighbours agree. This is why the architecture is a GNN and not an MLP, and it is a designed-in property of the data rather than a hopeful assertion.

The two other realism corrections:

1. **Bounded attacker source pools.** A real botnet controls a finite set of hosts. Drawing a fresh random source per packet had made `n_unique_src` a perfect give-away — unique sources equalled packet count, which nothing benign ever produces. Pool size now scales with intensity: `max(2, 3 + 180 · intensity2)`.
2. **Stealth-skewed attack intensity.** Intensities are drawn from a *squared* uniform, `0.08 + 0.92 · U2`, so most attacks are low-and-slow. Loud attacks are the easy case; a benchmark made mostly of them overstates every detector.

3.5 Orchestration: hospital, broker, runner, monitor

`twin/hospital.py` — `HospitalTwin`

One hospital: a device fleet, a gateway, an attack scheduler, a benign-event scheduler, and a clock. `tick(dt)` advances simulated time and returns every packet observed, in order: legitimate telemetry → benign-event mutations → attack mutations → gateway background → event injections → attack injections. Transport-agnostic by design.

`plan_attacks()` builds each hospital's schedule with three guaranteed properties: **coverage** (every attack class appears at every hospital, so the 7-way confusion matrix is well defined everywhere), **skew** (frequencies differ sharply — hospital *h* is dominated by class *h*, 45% dominant / 25% secondary / 30% spread), and **unambiguous labels** (attacks may run concurrently on *different* devices, but slots stay disjoint within one device's timeline — otherwise a window containing both a flood and a tampered vital could only carry one label).

`twin/broker.py`

Mosquito lifecycle. Detects an already-listening broker and reuses it; otherwise locates the binary (including Homebrew's `sbin` paths), writes `mosquitto/mosquitto.conf`, launches it, and waits for the port. Fails with an actionable message naming `brew install`, `apt install`, and the Docker command.

`twin/runner.py`

Two drivers over one core. `capture_offline()` runs every hospital headless and returns one labelled flow table — deterministic, fast, used by the dataset builder. `TwinRunner` publishes the same packets through a real broker in wall-clock time and **subscribes to a control topic** (`swarmdef/control`), so a presenter can fire an attack mid-demo by publishing JSON. Volumetric floods are published as periodic burst summaries carrying the true packet count — a 300 pkt/s flood would swamp the broker queue and starve the telemetry the demo needs to stay visible.

`twin/monitor.py`

Subscribing client that renders a refreshing colour terminal table of per-device packet rate, byte rate, last vitals and current label. It learns the device inventory from the telemetry topics themselves. Can optionally score windows with a trained detector live.

4. Step 3 — Dataset construction

4.1 The canonical 30-feature schema

`data/schema.py` is the contract the entire project is built on. Both data paths — the live twin and CIC-IoT2023 capture files — are projected onto *this* vector, in a fixed order that must never be reordered because models are trained on it.

Group	n	Features	Detects
volume	2	<code>packet_rate</code> , <code>byte_rate</code>	DDoS, flooding
size	6	<code>pkt_size_mean/std/min/max</code> , <code>payload_len_mean</code> , <code>header_len_mean</code>	fragmentation, firmware push
timing	4	<code>flow_duration</code> , <code>iat_mean</code> , <code>iat_std</code> , <code>burstiness</code>	beaconing, recon
entropy	2	<code>payload_entropy</code> , <code>topic_entropy</code>	encrypted / obfuscated payloads
protocol	8	<code>proto_{mqtt,tcp,udp}_ratio</code> , <code>flag_{syn,ack,rst}_ratio</code> , <code>qos_mean</code> , <code>retain_ratio</code>	spoofing, scanning
topology	7	<code>n_unique_{src,dst,topic}</code> , <code>conn_attempt_rate</code> , <code>subscribe_ratio</code> , <code>publish_ratio</code> , <code>dup_ratio</code>	recon, lateral movement
semantic	1	<code>vital_zscore_max</code>	MITM value tampering

The semantic feature is the MIoT-specific contribution: it asks whether the *clinical* value is physiologically plausible, which is a question no generic network IDS feature set poses. `burstiness` is the Goh–Barabási coefficient $B = (\sigma - \mu) / (\sigma + \mu) \in [-1, 1]$: -1 perfectly periodic (a C2 beacon), 0 Poisson, $+1$ bursty.

The immutable feature set — the threat model, encoded in the schema

Four features are declared `IMMUTABLE_FEATURES`: `conn_attempt_rate`, `n_unique_dst`, `vital_zscore_max`, `flag_rst_ratio`. The Step 6 GAN is mathematically forbidden from touching them. The reasoning: an attacker can pad packets or slow a flood down, but it *cannot un-tamper a vital it deliberately altered*, nor hide the connections it must open to function. Perturbing those would produce an adversarial vector that is no longer a working attack, and any evasion number from it would be meaningless.

Label vocabulary

7 classes: `BENIGN` (id 0) plus the six attack families in fixed order. Every row carries both a binary `label` and a 7-way `attack_type`, so binary and multi-class experiments read the same file.

4.2 Feature engineering — `data/features.py`

Raw packets are not a useful detector input: a single MQTT PUBLISH says almost nothing, and a DDoS is only visible as a *rate*. So packets are binned into **per-device 1-second flow windows** and summarised with the schema above.

The attribution rule (`attribute_device`) resolves in order: an explicit device endpoint wins; failing that, a device named in the MQTT topic — **this is what catches spoofed publishes**, whose source address is forged but whose topic still names the impersonated device; anything left over is segment traffic attributed to the gateway. Windows with fewer than 2 packets are dropped.

4.3 Normalisation — `data/scaler.py`

Not `StandardScaler`, and this is asked about. Flow features span orders of magnitude: a benign device publishes at 5 pkt/s while a flood reaches 400; byte rates differ by 10^4 . A plain z-score gives the flood $z \approx +30$ and squashes

every benign distinction to ~ 0 — destroying exactly the signal the *stealthy* attacks (MITM, Spoofing) depend on. So: **log1p** on the 16 heavy-tailed non-negative columns, then a **robust median/IQR** z-score (IQR/1.349, which makes it comparable to a standard deviation for Gaussian data), then clipped to ± 10 . Constant columns get scale 1.0 rather than dividing by zero. Saved to `data/processed/scaler.json` — 30 medians and 30 IQRs, frozen and reloadable.

4.4 Splitting — group-level, before the scaler is fitted

TWO LEAKAGE DEFENCES, BOTH DELIBERATE

1. The split happens before normalisation is fitted. The scaler sees only the train split, so test rows never influence the medians and IQRs. The verifier checks this empirically.

2. The split is at the graph level, not the row level. A whole (hospital, time-window) goes to one split. If neighbouring devices in the same window straddled a split boundary, a test node's features would reach the model during training through GNN message passing — a transductive setup that quietly inflates the score. Splitting whole windows makes evaluation **inductive**: the model is tested on graphs it has never seen any part of. That is also the honest setting for an IDS, which is deployed against future traffic.

Groups are stratified by the *rarest* attack class they contain, so a window holding the only Mirai sample cannot be lumped in with the benign bulk. Rows-level stratified splitting is also implemented (`split_mode: row`) for tabular-only comparisons.

4.5 Non-IID partitioning — `data/partition.py`

Federated learning is only interesting when clients disagree. Two partitioners:

- native** (default) — keep the twin's own hospital assignment. Skew arises organically: hospitals differ in *device fleet* as well as label mix. More realistic than relabelling a pooled set.
- dirichlet** — the standard synthetic benchmark. Per attack class, draw a Dirichlet(α) vector over hospitals and split accordingly; lower α means sharper skew. Required for CIC-IoT2023, which carries no hospital structure. Includes a top-up guard so a degenerate shard (one class, or < 32 rows) cannot break local training.

`earth_mover_skew()` turns "our split is non-IID" from an assertion into a **measurement**: the mean total-variation distance between each shard's class distribution and the pooled distribution. It is reported twice — overall, and **attacks-only**. The attacks-only figure is the informative one, because benign traffic is $\sim 75\%$ of every shard and leaving it in dilutes the very skew being quantified.

4.6 The dataset actually on disk

Property	Value
Source	<code>digital_twin</code> (CIC-IoT2023 not present, so auto-fallback fired)
Rows (flow windows)	12,600
Features	30
Attack fraction	24.92%
Split (group-level, stratified)	train 8,824 / val 1,272 / test 2,504
Hospitals	4 (3,600 / 3,000 / 3,000 / 3,000 rows)
Non-IID TV distance	0.284 across attack classes (0.078 including benign)
Seed	42, recorded in <code>dataset_meta.json</code>

Class counts

Class	BENIGN	MITM	DDoS	Spoofing	Recon	Mirai	FirmwareTamper
windows	9,460	784	653	577	475	361	290

Per-hospital threat mix — the non-IID evidence

Hospital	rows	attack %	BENIGN	DDoS	MITM	Firmware	Spoofing	Recon	Mirai	dominant
H0	3,600	25.4	2,684	440	226	30	54	74	92	DDoS
H1	3,000	25.6	2,233	95	370	99	45	78	80	MITM
H2	3,000	21.7	2,350	55	76	140	183	88	108	Spoofing/Firmware
H3	3,000	26.9	2,193	63	112	21	295	235	81	Spoofing/Recon

Each hospital sees a different dominant threat, and no hospital sees a representative sample of the landscape — which is precisely the condition federation is supposed to fix.

4.7 The CIC-IoT2023 loader — data/cic.py

Fully implemented, 256 lines. It maps CIC-IoT2023's 46 native flow columns and 33 attack labels onto the same 30-feature / 7-class schema. Highlights: DDoS-* and DoS-* collapse to DDoS; Mirai-* and DictionaryBruteForce to Mirai; Recon-* and VulnerabilityScan to Recon; MITM-ArpSpoofing to MITM; DNS_Spoofing to Spoofing; the code/injection families to FirmwareTamper. Features CIC does not measure (MQTT QoS, topic entropy, clinical plausibility) are set to zero, which is the correct value for a non-MQTT capture and keeps the vector dimension stable. Sampling is stratified per file so rare families survive the subsample.

Be precise about this in the review. The loader is written and the mapping is complete, but no CIC-IoT2023 CSVs are currently in data/raw/ , so it has never been run on real data. Every number in this document comes from the digital twin. Dropping CSVs into data/raw/cic_iot2023/ switches the pipeline over automatically — the source resolution is auto — but that run has not been done.

4.8 The verification suite — scripts/verify_dataset.py

Six checks, each printing PASS/FAIL, with a non-zero exit code on any failure:

- 1. **schema** — all 30 features and 5 metadata columns present; every value finite.
- 2. **splits** — train/val/test all present and every one of the 7 classes appears in test.
- 3. **leakage** — train medians are centred near 0 ($|median| < 0.15$) while test medians are merely close. Identical medians would be evidence the scaler saw the test set.
- 4. **non-IID** — measured attack-class TV distance > 0.15 , plus the dominant threat per hospital.
- 5. **separability** — a deliberately weak logistic-regression probe must beat the majority-class baseline by > 5 points. If a linear model can find signal, the features are sound, and any later GNN failure is a modelling bug rather than a broken dataset.
- 6. **shards** — every hospital has both classes and ≥ 50 training rows, i.e. is actually trainable.

It also prints the full per-hospital threat table and per-class linear-probe recall. The README records 6/6 passing at $\sim 96.5\%$ linear-probe accuracy with per-class recall 0.93–1.00 and a 3.7% benign false-positive rate — a realistic IDS operating point, with real headroom left for the later steps.

5. Step 4 — Detector architectures

Three architectures over the identical 30-D features, identical splits, identical training loop — so the comparison is genuinely controlled and any difference is attributable to architecture alone.

5.1 The three models — `detector/models.py`

MLPDetector — the control

Per-window classifier with **no context at all**. It exists so that any gain by the GNN or Transformer is attributable to context. 6,530 parameters (binary).

TransformerDetector — temporal context

Attends over a device's recent flow windows to classify the current one. Fixed sinusoidal positional encoding (learned embeddings would mean different things across clients with different sequence lengths). 69,186 parameters. Self-attention is **written out from plain `nn.Linear` layers** rather than using `nn.MultiheadAttention` — Opacus cannot attach per-sample gradient hooks to PyTorch's fused packed-projection weight, so using the built-in would have blocked Step 7 entirely.

GNNDetector — spatial context, the primary model

GraphSAGE over the device graph, classifying each device-window node. 18,690 parameters. Supports `sage`, `gcn`, and `gat` convolutions.

WHY SAGECONV SPECIFICALLY, AND NOT GCN

SAGEConv keeps a **separate weight for a node's own features and for the mean of its neighbours**. That separation is exactly what this task needs: the model must be able to express "*my DUP rate is high and my neighbours' is not*" — which is the MITM-versus-NetworkCongestion decision. A convolution that averaged self and neighbours together (GCN) would blur precisely the contrast being tested.

Three constraints carried backwards from later steps

1. **LayerNorm, never BatchNorm.** Opacus cannot compute per-sample gradients through BatchNorm because it mixes information across the batch — and BatchNorm is also the classic source of silent divergence in FedAvg when clients have non-IID batches. Using LayerNorm from the start means Steps 5 and 7 need no architectural rewrite.
2. **Flat, ordered parameter vectors.** `get_parameters()` / `set_parameters()` exchange plain NumPy arrays in fixed order — exactly what Flower transmits and what the Byzantine aggregators operate on.
3. **No in-place activations.** In-place ops break Opacus' gradient hooks.

A residual skip that is load-bearing

`GNNDetector` has `self.skip = nn.Linear(in_dim, hidden)` feeding straight from the raw features. Without it, a node with no informative neighbours would depend entirely on message passing. It also makes the model usable when `edge_index` is empty — which is not an error but a meaningful input: **Step 6's adversarial samples are single windows the attacker controls, with no ward context attached.**

5.2 Graph construction — `detector/graph.py`

One graph per (hospital, time-window). Nodes are that hospital's devices during that second, each carrying its 30-D vector and its own label. The task is therefore **node classification**: which devices are under attack right now.

Topology	Edges	Rationale
star	every clinical device ↔ the gateway	The real MQTT path — devices don't talk peer-to-peer, they publish through the broker, which the gateway observes.
peer	devices of the same type ↔ each other	Two infusion pumps should behave alike; divergence between them is evidence.
knn	k most behaviourally similar devices	Lets the graph adapt when physical topology is unknown, as for CIC captures.

Default is `star_peer` with `graph_knn=4`. `build_sequences()` provides the parallel sequence view for the Transformer: per-device sliding windows of 8 consecutive flow vectors, labelled by the *final* step (the causal setting an online IDS actually faces), built strictly inside one split so a window never spans the train/test boundary.

5.3 One loader interface for three architectures — `detector/data.py`

The GNN consumes batched graphs, the Transformer consumes sequences, the MLP consumes single windows. Rather than special-casing architecture at every call site, every loader yields the same 3-tuple (`inputs`, `edge_index_or_None`, `targets`). This is what lets Steps 5–8 swap the detector without touching the federated, DP or aggregation code.

`class_weights()` computes inverse-frequency loss weights. Without them the model can score 75% by predicting BENIGN everywhere and recall on Mirai and Recon — the classes that matter most — collapses.

5.4 Training — `detector/train.py`

One `train_model/evaluate` pair, reused by the centralised baseline, every federated client's local update, adversarial retraining, and (when built) DP-SGD. Sharing one implementation is what makes those results comparable — a separate loop per experiment would introduce differences unrelated to the mechanism under study. AdamW, cross-entropy with class weights, early stopping on validation F1 with best-weight restoration. It accepts a pre-built `optimizer` so Step 7 can pass an Opacus-wrapped one and reuse the loop unchanged.

5.5 Step 4 results — binary detection (test split, n = 2,504)

Architecture	params	accuracy	F1	precision	recall	FPR	AUC	train s
MLP	6,530	0.9788	0.9576	0.9214	0.9967	0.0268	0.9975	1.2
Transformer	69,186	0.9764	0.9527	0.9181	0.9900	0.0278	0.9970	16.2
GNN / SAGE	18,690	0.9804	0.9605	0.9311	0.9917	0.0231	0.9986	1.3
GNN / GAT	12,930	0.9780	0.9558	0.9238	0.9900	0.0257	0.9979	1.8

Confusion matrix, GNN/SAGE: TP 595, FP 44, TN 1,860, FN 5.

5.6 Step 4 results — 7-way attack typing

Architecture	7-way acc	binary acc	F1	BENIGN	DDoS	MITM	Firmware	Spoofing	Recon	Mirai
MLP	0.9732	0.9764	0.9528	0.971	0.944	0.986	1.000	0.983	0.988	1.000
Transformer	0.9708	0.9724	0.9448	0.969	0.992	0.944	0.966	0.991	0.988	0.986
GNN / SAGE	0.9736	0.9740	0.9484	0.967	0.992	0.986	1.000	1.000	0.988	1.000
GNN / GAT	0.9681	0.9700	0.9400	0.967	0.992	0.903	1.000	0.983	0.988	1.000

Per-class recall on the test split. GNN/SAGE is the only model at ≥ 0.967 on every one of the seven classes.

THE HONEST READING OF STEP 4 – DO NOT OVERCLAIM THIS

On **clean** traffic, the GNN's margin over the MLP is small: +0.16 accuracy points, +0.29 F1 points, and the MLP actually has marginally higher binary recall. All four architectures are within ~ 1 point of each other. If a panel member says "your GNN barely beats a 2-layer MLP", **that is correct on clean data and you should agree.**

The defensible answer is that Step 4 is a *ceiling check*, not the argument. The GNN does win on the metrics that matter operationally — **lowest false-positive rate (2.31% vs 2.68%), highest AUC, and uniquely ≥ 0.967 recall on all seven classes** — and it does so with $3.7\times$ fewer parameters and $12\times$ less training time than the Transformer. The real separation appears under adversarial pressure in Step 6, and in the ability to use ward context that no per-window model has access to.

6. Step 5 — Federated learning

6.1 HospitalClient — federated/client.py

One client owns one shard and never exposes it. Each round it receives global parameters, trains locally for `local_epochs` (2), and returns **only** the updated parameters plus a sample count.

Crucially, a "malicious hospital" is a **configuration of a normal client, not a separate code path**. The server cannot tell them apart — which is the entire point of Byzantine robustness. Four poisoning modes are implemented in `corrupt()`:

Attack	What the client uploads
<code>sign_flip</code>	$g - \text{scale} \cdot (p - g)$ — the negation of the honest update, pulling the global model away from the optimum every step.
<code>scale</code>	$g + \text{scale} \cdot (p - g)$ — norm-scaling / model replacement: amplify until the update dominates the average.
<code>gauss</code>	Random noise of comparable magnitude to a real update.
<code>label_flip</code>	Data poisoning, applied at construction: the client trains on inverted labels, so its update is a <i>plausible</i> gradient pointing the wrong way. Far harder for a distance-based rule to reject than a scaled update, because its norm looks entirely normal.

The client also carries two extension points used by later steps: `self.adversary` (Step 6) and `self.privacy_engine` (Step 7). Both are wired into `fit()` already — Step 7 needs the engine written, not the client changed.

6.2 Five aggregation rules — federated/aggregators.py

THE ONE-SENTENCE REASON PLAIN AVERAGING FAILS

FedAvg computes a **mean**, and a mean has an asymptotic breakdown point of **zero**: one client sending sufficiently large values can move the average anywhere it likes, no matter how many honest clients there are. Every robust rule below replaces the mean with an estimator that ignores extreme values — either by *selecting* a representative update (Krum) or by *discarding the tails* coordinate by coordinate (trimmed mean, median).

Rule	Mechanism and reference
<code>fedavg</code>	Sample-weighted mean (McMahan et al. 2017). Optimal when everyone is honest; no protection when someone is not.
<code>krum</code>	Blanchard et al. 2017. Scores each client by the sum of squared distances to its $n-f-2$ nearest neighbours, keeps the lowest scorer. Intuition : honest updates cluster because they estimate the same gradient, so an honest client has many close neighbours; a poisoned update must sit far from that cluster to do damage — and an attacker close enough to score well is, by construction, too close to matter. Requires $n > 2f + 2$.
<code>multikrum</code>	Averages the $n-f$ best-scoring updates instead of just one, recovering the variance reduction single-Krum throws away.
<code>trimmed_mean</code>	Yin et al. 2018. Per parameter, discard the top and bottom <code>trim_ratio</code> of client values and average the rest. Never discards a whole honest client — only its outlying coordinates. Falls back to the median when there are too few clients to trim both tails.
<code>median</code>	Coordinate-wise median. Maximally robust — 50% breakdown point — at the cost of discarding magnitude information from every honest client.

6.3 FederatedSimulation — federated/simulation.py

```
for each round:
    broadcast  $\theta_{\text{global}}$  to the selected hospitals
    each hospital trains locally on data that never leaves it           [Step 5]
    ... optionally hardened against GAN-generated evasions             [Step 6]
    ... optionally clipped + noised before upload                       [Step 7 – not built]
    the server aggregates the uploads, robustly                         [Step 8]
    the new global model is evaluated on the pooled held-out test set
```

Every experiment is scored on **exactly the same** pooled test split, so federated, DP and Byzantine runs are directly comparable to the Step 4 baseline. Byzantine node identity is fixed up front, so the same hospitals are malicious every round. `per_hospital_report()` answers the federated-specific question a pooled baseline cannot: *does the shared model actually help the hospital that contributed least, or does it only serve the majority?*

6.4 The real Flower deployment — federated/flower_app.py

The custom simulator runs the experiments — deterministic, fast, easy to instrument, which is what the Step 6–8 ablations need. This module proves the *same system* runs under Flower's actual client/server protocol, serialisation and strategy machinery: the path a production deployment would take. Both share the identical `HospitalClient` and the identical aggregation functions, so a simulation result is a result about this system, not about the harness.

`RobustStrategy` subclasses Flower's `FedAvg` and overrides **only** `aggregate_fit`, so the Byzantine defence under test is literally the same function in both harnesses — there is no second implementation that can drift.

A REAL BUG THAT WAS FOUND, DIAGNOSED AND DOCUMENTED IN THE CODE

An earlier version of `client_fn` closed over pre-built `HospitalClient` objects holding torch models and DataFrames. Flower's Ray backend runs each client in a separate worker process, so those objects could not be serialised and shipped — the workers died on startup and the run silently produced a global model **frozen at its initialisation**, reporting majority-class accuracy every round. Clients are now constructed on demand inside the worker from the plain config, which is also how a real deployment works, since each hospital constructs its own client locally. A guard was added that logs an explicit error if accuracy is identical across all rounds.

6.5 Step 5 results — FedAvg over 4 hospitals, GNN detector, 15 rounds

round	accuracy	F1	precision	recall	FPR	AUC	evasion	loss
1	0.8606	0.7165	0.6989	0.7350	0.0998	0.9031	0.2650	0.444
2	0.9373	0.8706	0.8613	0.8800	0.0446	0.9634	0.1200	0.279
4	0.9617	0.9220	0.9000	0.9450	0.0331	0.9857	0.0550	0.140
8	0.9728	0.9452	0.9156	0.9767	0.0284	0.9945	0.0233	0.094
12	0.9752	0.9502	0.9177	0.9850	0.0278	0.9956	0.0150	0.076
15	0.9740	0.9476	0.9173	0.9800	0.0278	0.9966	0.0200	0.068

THE HEADLINE FEDERATED NUMBER

Centralised (pooled, illegal in practice): 0.9804 accuracy / 0.9605 F1.

Federated (best, round 12, no data ever leaves a hospital): 0.9752 / 0.9502.

Privacy cost of federation: –0.0052 accuracy — about half a percentage point.

Convergence is clean and monotone: 0.861 → 0.937 → 0.962 → 0.975 over twelve rounds, with training loss falling 0.444 → 0.076 and false-positive rate falling from 9.98% to 2.78%.

7. Step 6 — The GAN adversarial engine

7.1 The threat model, stated exactly

The adversary already controls an attacking host inside the ward and wants its traffic classified BENIGN. It may reshape *how* the attack looks on the wire — pad packets, slow a flood down, spread it over more connections — but it may not change what the attack fundamentally does. So the generator produces a bounded perturbation, not an arbitrary sample:

```
x_adv = x + ε · tanh(G(x, z)) · mutable_mask
```

Three constraints, each a claim that must be defensible:

1. **Bounded** ($L^\infty \leq \epsilon$). An unbounded perturbation could simply overwrite the sample with a benign one, which proves nothing about the detector.
2. **Masked**. Only `schema.mutable_mask()` features may move. `vital_zscore_max`, `conn_attempt_rate`, `n_unique_dst` and `flag_rst_ratio` are frozen — an attacker cannot un-tamper a vital it deliberately altered, nor hide the connections it must open.
3. **Directionally honest**. The perturbation is applied in the same normalised space the detector consumes, so the reported evasion rate is measured against exactly the features the model sees.

This is a **white-box** attack: the generator differentiates through the current detector. That is the strongest reasonable adversary for evaluating a defence, and it is what "zero-day-style" means operationally — the attacker adapts to *this* detector, not to a fixed signature list.

`budget_report()` verifies compliance on every run and the results record it: `within_budget: true`, `frozen_features_untouched: true`. Measured L^∞ at $\epsilon = 1.0$ was 0.9939 — right at the budget, never over it.

7.2 Training the generator — `gan/engine.py`

The loss is

```
L_G = CrossEntropy( D(x_adv), BENIGN ) + λ · ||x_adv - x||_2      (λ = 0.05)
```

The detector plays the discriminator role from the report's min-max objective. **No separate discriminator network is trained**, and this is a deliberate design choice worth stating: the quantity of interest is evasion of the *deployed* detector, not sample realism against an auxiliary critic. The sparsity term keeps the perturbation small inside the allowed budget — an attack that spends its whole budget is easier for adversarial training to learn, and a real adversary prefers the smallest change that works. The detector is frozen during generator training (it is the target, not the student).

`_detector_logits()` handles the architecture mismatch honestly: a GNN gets an **empty edge index** and a Transformer gets a **length-1 sequence**, because the attacker controls its own traffic, not the behaviour of the ward around it.

7.3 Adversarial training — `gan/adv_train.py`

`ClientAdversary` plugs into `HospitalClient.adversary`. Each round, per hospital:

1. Skip entirely for the first `warmup_rounds` (2) — attacking a model that has not converged measures nothing about the defence and destabilises early rounds.
2. Train the generator against the **current global detector**.
3. Generate evasive variants of that hospital's own attack windows and label them as what they really are: **attacks**. This is what teaches the detector the blind spot.

4. Mix with real training data at `adv_train_ratio` = 0.5 and retrain.

`evaluate_under_attack()` perturbs **only the attack half** of the test set — an attacker has no reason to perturb traffic it does not control — so the reported accuracy is the operationally meaningful number. During federated runs, the adversary is **retrained fresh against the current global model every round**; reusing an older generator would understate the threat, since a real attacker re-optimises against whatever is deployed.

7.4 Step 6 results — degradation and recovery ($\epsilon = 1.0$, 10 rounds, 600 attack samples)

Detector	Traffic	accuracy	F1	attack recall	evasion rate
undefended	clean	0.9665	0.9341	0.9917	0.0083
	GAN-evasive	0.8998	0.7733	0.7133	0.2833
adversarially trained	clean	0.9760	0.9520	0.9917	0.0083
	GAN-evasive	0.9681	0.9350	0.9583	0.0433

Quantity	Value	Reading
GAN degrades attack recall by	−0.2783	28.3% of real attacks slip past the undefended detector
Adversarial training recovers	+0.2450	only 4.3% still evade — an 85% reduction in evasion
Robustness tax on clean traffic	−0.0096	negative — the defended model is <i>better</i> on clean traffic too

THE STRONGEST SINGLE RESULT IN THE PROJECT

Adversarial training here has a **negative robustness tax**. Clean accuracy went *up* from 0.9665 to 0.9760 and clean F1 from 0.9341 to 0.9520 while evasion fell from 28.3% to 4.3%. The usual textbook trade-off — robustness bought with clean accuracy — did not appear at this budget. The most likely explanation, and the one to give: the GAN's bounded, mask-respecting perturbations act as a **targeted data augmentation along exactly the directions the detector was weakest in**, and the model had enough capacity to absorb them. Present this as an observed result with a proposed mechanism, not as a proven law.

7.5 The budget sweep — why a single evasion number would be meaningless

A single evasion figure says nothing without knowing how much power the attacker was given. So evasion is measured across five L^∞ budgets against both detectors:

ϵ	undef. accuracy	undef. attack recall	undef. evasion	def. accuracy	def. attack recall	def. evasion	L^∞ used
0.25	0.9593	0.9617	3.83%	0.9752	0.9883	1.33%	0.249
0.50	0.9473	0.9117	9.00%	0.9744	0.9850	1.50%	0.498
0.75	0.9365	0.8667	13.67%	0.9724	0.9767	2.17%	0.746
1.00	0.9022	0.7233	27.83%	0.9712	0.9717	2.83%	0.997
1.50	0.8395	0.4617	54.00%	0.9573	0.9133	8.67%	1.493

The undefended detector degrades sharply and non-linearly — at $\epsilon = 1.5$ it catches fewer than half of all attacks. The defended detector degrades gracefully, still catching 91.3% at the same budget. Budget compliance was verified true at every point.

Why ϵ defaults to 0.5, not 0.15

An earlier default of $\epsilon = 0.15$ was calibrated away and the reasoning is written into `config.py`: 0.15 is far below the noise floor of these features and produced <1% evasion against every architecture — which measures the *budget*, not the defence. $\epsilon = 0.5$ means half an inter-quartile range per feature, which is a realistic amount of slack for an attacker padding packets or pacing a flood.

7.6 The federated-adversarial coupling, made measurable

Both Step 6 runs are federated: 4 hospitals, FedAvg, GNN, 10 rounds. Each hospital runs **its own generator against its own copy of the detector**, finds blind spots specific to *its* threat mix, and hardens against them. Aggregation then propagates every hospital's hardening to all the others. Since H0 is DDoS-dominated, H1 MITM-dominated, H2/H3 Spoofing-dominated, the four adversaries search different regions of the feature space — and the merged model inherits all four searches while no hospital ever shared a packet. That is the project's actual security claim, and Step 6 is where it stops being a slogan.

8. Complete module reference

Every Python file in the repository, what it contains, and why it exists. 48 files, 6,720 lines.

Top level

File	lines	Purpose
<code>swarmdef/__init__.py</code>	19	Package docstring naming the eight sub-packages and the pipeline.
<code>swarmdef/config.py</code>	256	Typed dataclass config for the whole pipeline: <code>TwinConfig</code> , <code>DataConfig</code> , <code>DetectorConfig</code> , <code>GANConfig</code> , <code>PrivacyConfig</code> , <code>FederatedConfig</code> , <code>EvalConfig</code> . Load order: dataclass defaults → YAML → dotted CLI overrides (<code>--set a.b=c</code>). <code>propagate_seed()</code> pushes the root seed everywhere and keeps hospital/client counts consistent across layers. Every run is fully described by one <code>Config</code> , serialised alongside its results.
<code>configs/default.yaml</code>	—	The default experiment: seed 42, 4 hospitals, GNN, 15 rounds, FedAvg, ϵ 0.5, DP disabled.
<code>requirements.txt</code>	—	Pinned minimum versions with an inline comment explaining each package's role.

`swarmdef/utils/` — foundations

File	lines	Purpose
<code>seed.py</code>	39	<code>set_seed()</code> seeds Python, NumPy and torch (plus cuDNN determinism); <code>resolve_device()</code> picks CUDA → CPU, deliberately skipping MPS.
<code>logging.py</code>	67	Colour console logging, optional file handler, <code>banner()</code> section headers for the step-by-step demo output, and silencing of chatty third-party loggers.
<code>metrics.py</code>	96	<code>DetectionMetrics</code> (accuracy, precision, recall, F1, FPR , full confusion counts), <code>compute_metrics()</code> , <code>attack_success_rate()</code> — the headline evasion number — and a tie-corrected rank-based <code>roc_auc()</code> . All computed here once so every experiment reports identical definitions.

swarmdef/twin/ — Step 2

File	lines	Purpose
packet.py	109	<code>Packet</code> wire record + Shannon entropy; <code>FlowWindow</code> with the any-attack-packet labelling rule.
devices.py	281	<code>Vital</code> (Ornstein–Uhlenbeck), five clinical device classes, <code>build_fleet()</code> , and the <code>Gateway</code> with its five-way benign background traffic mix.
attacks.py	392	Six attack scenarios with <code>mutate</code> / <code>inject</code> hooks, plus the <code>build_attack()</code> factory.
events.py	315	Six benign confounders, <code>EVENT_TARGETS</code> mapping each to eligible device types, and <code>build_event()</code> .
hospital.py	304	<code>HospitalTwin</code> (fleet + schedulers + clock + <code>tick</code>), <code>ScheduledAttack</code> , <code>ATTACK_TARGETS</code> , and <code>plan_attacks()</code> with its coverage / skew / disjointness guarantees.
broker.py	123	Mosquito detection, config generation, launch, readiness polling, clean shutdown; <code>BrokerHandle</code> is a context manager.
runner.py	300	<code>capture_offline()</code> (headless, deterministic) and <code>TwinRunner</code> (live MQTT, wall-clock paced, control-topic attack triggering, flood summarisation). <code>trigger_attack()</code> is the one-shot CLI helper.
monitor.py	223	<code>LiveMonitor</code> : subscribing client, per-device rolling stats over a 3 s sliding window, refreshing colour terminal table.

swarmdef/data/ — Step 3

File	lines	Purpose
schema.py	88	The 30 feature names in fixed order, <code>IMMUTABLE_FEATURES</code> , the 7-class label vocabulary and id maps, metadata columns, <code>mutable_mask()</code> .
features.py	190	<code>attribute_device()</code> , <code>window_features()</code> (the 30 computations), <code>FlowFeatureExtractor</code> binning packets to per-device windows and materialising the labelled frame.
scaler.py	97	<code>FlowScaler</code> : log1p on 16 heavy-tailed columns, robust median/IQR z-score, ± 10 clip, JSON save/load.
partition.py	151	<code>native_partition</code> , <code>dirichlet_partition</code> (with degenerate-shard top-up), <code>skew_report()</code> , <code>earth_mover_skew()</code> .
cic.py	256	CIC-IoT2023: 33-label $\rightarrow$ 6-family map, CSV discovery, <code>project_to_schema()</code> mapping 46 native columns onto the 30-D vector, stratified per-file sampling.
build.py	278	The end-to-end builder: source resolution $\rightarrow$ raw load $\rightarrow$ group/row split $\rightarrow$ scaler fit on train only $\rightarrow$ non-IID partition $\rightarrow$ persist shards, pooled CSV, scaler and <code>dataset_meta.json</code> . Plus <code>load_hospital</code> , <code>load_pooled</code> , <code>split_xy</code> .

swarmdef/detector/ — Step 4

File	lines	Purpose
models.py	231	<code>Detector</code> base with Flower-format parameter exchange; <code>MLPDetector</code> , <code>TransformerDetector</code> (with Opacus-safe hand-written attention), <code>GNNDetector</code> (SAGE/GCN/GAT + residual skip); <code>build_detector()</code> factory used by every entry point so all steps build identical models.
graph.py	206	<code>build_edges()</code> (star / peer / full / knn), <code>build_graphs()</code> (one PyG <code>Data</code> per hospital-window), <code>graph_stats()</code> , <code>build_sequences()</code> for the Transformer.
data.py	122	<code>GraphBatchLoader</code> , <code>TensorBatchLoader</code> , the unified <code>make_loader()</code> , and inverse-frequency <code>class_weights()</code> .
train.py	155	<code>predict</code> , <code>evaluate</code> (binary metrics + per-class recall + AUC + evasion), <code>train_model</code> with early stopping and best-weight restore, and <code>TrainHistory</code> for the learning-curve figures.

swarmdef/gan/ — Step 6

File	lines	Purpose
generator.py	94	<code>PerturbationGenerator</code> : bounded, masked perturbation network plus <code>budget_report()</code> which verifies L_∞ compliance and that frozen features were untouched.
engine.py	134	<code>AdversarialEngine</code> : <code>fit()</code> (train generator vs frozen detector), <code>evaluate()</code> (evasion before/after + budget check), <code>generate()</code> , and the architecture-aware <code>_detector_logits()</code> .
adv_train.py	118	<code>ClientAdversary</code> (per-hospital adversary with warmup gating and loader augmentation) and <code>evaluate_under_attack()</code> (perturbs only the attack half of a test split).

swarmdef/federated/ — Steps 5 & 8

File	lines	Purpose
client.py	188	<code>HospitalClient</code> , <code>ClientReport</code> , the four Byzantine poisoning modes, and the adversary / privacy hooks.
aggregators.py	178	<code>flatten</code> / <code>unflatten</code> , <code>pairwise_sq_distances</code> , and the five rules: <code>fedavg</code> , <code>krum</code> , <code>multikrum</code> , <code>trimmed_mean</code> , <code>coordinate_median</code> , with a kwarg-filtering <code>aggregate()</code> dispatcher.
simulation.py	224	<code>FederatedSimulation</code> : client construction, round loop, robust aggregation, per-round evaluation, optional adversarial tracking, per-hospital reporting, experiment logging.
flower_app.py	203	<code>RobustStrategy</code> (FedAvg subclass calling our aggregators), <code>_NumPyClient</code> , <code>client_fn</code> , and <code>run_flower_simulation()</code> driving flwr's <code>ServerApp</code> / <code>ClientApp</code> .

swarmdef/eval/ and swarmdef/privacy/

File	lines	Purpose
eval/logger.py	77	<code>RoundRecord</code> + <code>ExperimentLogger</code> : one schema for every experiment, writing paired CSV and JSON (with run metadata) so figures and tables read from one place.
eval/style.py	92	Shared matplotlib styling: an eight-hue colourblind-checked categorical palette assigned in fixed order and never cycled (so "the blue line" means the same thing in every figure), reserved status colours, light/dark surfaces.
eval/plots.py	282	Six figure generators: <code>architecture_comparison</code> , <code>learning_curves</code> , <code>federated_vs_central</code> , <code>adversarial_impact</code> , <code>privacy_tradeoff</code> (ready for Step 7), <code>byzantine_comparison</code> (ready for Step 8), <code>non_iid_heatmap</code> .
privacy/__init__.py	1	Empty package — Step 7 has not been written. Only the docstring exists.

scripts/ — entry points

Script	lines	Purpose
run_twin.py	107	Four subcommands: <code>stream</code> (live MQTT, auto-starts Mosquitto), <code>monitor</code> (live terminal view), <code>attack</code> (trigger an attack mid-stream), <code>capture</code> (headless to CSV).
build_dataset.py	48	Step 3 entry point with <code>--force</code> and dotted overrides; prints the metadata summary.
verify_dataset.py	128	The 6-check suite plus the threat-mix and per-class-detectability tables; exits non-zero on failure.
train_baseline.py	148	Step 4: trains MLP / Transformer / GNN-SAGE / GNN-GAT on the pooled set, binary or 7-way, saves four <code>.pt</code> checkpoints, JSON results and four figures.
train_federated.py	171	Step 5: FedAvg run, comparison against the Step 4 reference, per-hospital report, and an optional <code>--compare-local</code> isolated-hospital baseline.
train_federated_flower.py	60	Step 5 deployment path: the same loop under real Flower.
train_adversarial.py	162	Step 6: the four-phase experiment (federate → attack → federate with adversarial training → attack again) plus the five-point budget sweep.

THE `--COMPARE-LOCAL` FLAG IS WORTH KNOWING ABOUT

`isolated_baseline()` in `train_federated.py` trains each hospital **alone** on its own shard and scores it on the *global* test set. Its docstring makes the sharpest argument in the project: *"Beating a pooled baseline is not the goal (pooling is illegal here); what matters is whether a hospital does better by joining the swarm than by training in isolation on the narrow slice of the threat landscape it happens to observe."* If a panel member asks what federation buys, that is the comparison — and running `--compare-local` before the review would give you the number.

9. All measured results, in one place

9.1 Artefacts on disk

Path	Contents
data/processed/pooled.csv	12,600 rows × 36 columns (30 features + 5 metadata + split tag)
data/processed/scaler.json	Frozen normalisation: log mask, 30 medians, 30 IQR scales
data/processed/dataset_meta.json	Full provenance: source, splits, class counts, skew stats, seed
data/hospitals/hospital_0..3.csv	The four federated shards (3,600 / 3,000 / 3,000 / 3,000)
results/models/baseline_*.pt	4 checkpoints: mlp, transformer, gnn_sage, gnn_gat — each stores state dict, arch, conv, n_classes and the full config
results/logs/step4_baseline_{binary,multiclass}.json	Architecture comparison, both tasks
results/logs/step5_fedavg_gnn.{csv,json}	15 federated rounds + run metadata
results/logs/step5_flower.csv	Flower deployment run (see §12 — this run did not converge)
results/logs/step6_{undefended,defended}.{csv,json}	10 rounds each, with per-round under-attack scores
results/logs/step6_adversarial.json	Degradation/recovery summary + the 5-point budget sweep
results/figures/*.png	11 figures (listed below)

9.2 The eleven figures generated

Figure	Shows
step3_non_iid_heatmap.png	Per-hospital threat mix as a sequential heatmap — the non-IID evidence
step4_architectures_f1_binary.png step4_architectures_accuracy_binary.png	Horizontal bars comparing the four architectures, binary task
step4_architectures_f1_multiclass.png step4_architectures_accuracy_multiclass.png	Same, 7-way task
step4_architectures_f1.png, step4_architectures_acc.png	Earlier-run versions of the same charts
step4_learning_curves_{binary,multiclass}.png	Validation F1 per epoch, one line per architecture
step5_federated_vs_central_gnn.png	Per-round federated accuracy against the pooled-data reference line
step6_adversarial_impact.png	Clean vs under-GAN-attack vs after-retraining, per round

9.3 The single comparison table to have memorised

Setting	accuracy	F1	attack recall	FPR	Note
Centralised GNN (pooled data)	0.9804	0.9605	0.9917	0.0231	the illegal-in-practice ceiling
Federated GNN, FedAvg (best round)	0.9752	0.9502	0.9850	0.0278	no data leaves a hospital
Federated, undefended, under GAN	0.8998	0.7733	0.7133	—	28.3% of attacks evade
Federated + adversarial training, clean	0.9760	0.9520	0.9917	—	better than undefended clean
Federated + adversarial training, under GAN	0.9681	0.9350	0.9583	—	4.3% evade

Cost of federation vs pooling: -0.52 accuracy points. Cost of the GAN attack on an undefended model: -8.7 accuracy points, -27.8 recall points. Recovery from adversarial training: $+24.5$ recall points, with clean accuracy also improving.

10. Design decisions, and how to defend each one

Decision	Defence
One 30-D schema for the twin <i>and</i> CIC-IoT2023	A model trained on real captures can be evaluated on live twin traffic without any change to the detector, and vice-versa. The alternative — two feature sets — would make the two sources incomparable and the twin unfalsifiable.
Window-level labels: "any attack packet ⇒ malicious"	Standard IDS convention. It forces the detector to spot a hostile <i>minority</i> inside otherwise benign traffic, which is the realistic case — a single injected packet compromises the window.
A Gateway pseudo-device	Recon and Spoofing address no specific device. Without a gateway node they were being silently dropped and two whole attack classes vanished from the dataset. It also needs its own benign baseline, otherwise the node is trivially separable.
Attacks concurrent across devices, disjoint per device	Maximises attack density while keeping window labels unambiguous. Overlapping attacks on one device would produce a window that is both a flood and a tampered vital, and the multi-class metrics would silently degrade.
The scaler is fitted centrally on the train split	Acknowledged as a mild relaxation of data isolation, and standard in the FL-IDS literature. The alternative — per-client scalers — makes each client's weights describe a <i>different input space</i> , so averaging them is not meaningful. What is shared is 30 medians and 30 IQRs, not records; a real deployment would obtain them in one DP-protected secure-aggregation round. Have this answer ready — it is the most likely methodological challenge.
<code>IMMUTABLE_FEATURES</code> in the schema	The GAN may only perturb features an attacker could actually control without breaking the attack's function. Without this, "evasion" would just mean "the optimiser overwrote the sample with a benign one", which measures nothing.
Native (not Dirichlet) partition by default	The twin's hospitals already differ in device fleet <i>and</i> threat mix — non-IID at the source, more realistic than relabelling a pooled set. Dirichlet is implemented and is used for CIC-IoT2023, which has no hospital structure of its own.
Group-level (inductive) splitting	Row-level splitting would let a test node's features reach the model during training via GNN message passing — transductive leakage that inflates the score. Whole time-windows go to one split, so the model is tested on graphs it has never seen any part of. That is also the deployment reality for an IDS.
LayerNorm everywhere, no BatchNorm	Two independent reasons: Opacus cannot compute per-sample gradients through BatchNorm, and BatchNorm is a known cause of silent FedAvg divergence with non-IID client batches. Choosing it in Step 4 means Steps 5 and 7 needed no rewrite.
No separate discriminator network in the GAN	The detector <i>is</i> the discriminator. The quantity of interest is evasion of the deployed detector, not sample realism against an auxiliary critic. Training a second critic would optimise the wrong objective.
The adversary is retrained fresh every round	Reusing a stale generator would understate the threat. A real attacker re-optimises against whatever is currently deployed, so the defence must be measured against an adversary that has adapted to the current global model.
Two federated harnesses (own simulator + Flower)	The simulator is deterministic and instrumentable, which the ablations need; Flower proves the same clients and aggregation rules run under a real framework's protocol. Both call the <i>same</i> <code>aggregate()</code> function, so there is no second implementation that can drift.
A benchmark deliberately made harder	The first dataset was 99.9%-solvable by logistic regression, which would have made every downstream plot a flat line at 1.0. It was fixed with three <i>realism</i> corrections — benign confounders, bounded attacker source pools, stealth-skewed intensity — not with artificial label noise. This is the difference between a benchmark and a demo.

11. Anticipated panel questions, with answers

"This is synthetic data. Why should we believe any of these numbers?"

Three answers, in order. **First**, the twin is not a random-number generator: vitals follow bounded Ornstein–Uhlenbeck processes, entropy is computed from real serialised payloads, and each attack has a mechanistically distinct signature. **Second**, the benchmark was deliberately made harder rather than easier — benign confounder events create genuine class overlap, attacker source pools are bounded, and intensities are stealth-skewed, taking linear-probe accuracy from 99.9% down to ~96.5%. **Third**, the CIC-IoT2023 loader exists and maps 46 native columns and 33 labels onto the identical schema, so the same detector can be evaluated on real captures without a code change. That validation run has not been done yet, and I would not claim otherwise.

"Your GNN barely beats the MLP. Why not just use the MLP?"

On clean traffic, correct — all four architectures are within about a point. But the GNN has the lowest false-positive rate (2.31% vs 2.68%), the highest AUC, and is the only model with ≥ 0.967 recall on all seven classes, at $3.7\times$ fewer parameters and $12\times$ less training time than the Transformer. More importantly, the dataset contains a discrimination the MLP *structurally cannot* make: CodeBlue and MITM are constructed to be identical in the victim device's own feature vector, and only differ in whether the surrounding ward reacts. That evidence is not in the per-window vector. Step 4 is a ceiling check; the architecture argument rests on that designed-in property and on behaviour under adversarial pressure.

"Fitting the scaler on pooled data breaks federated privacy."

It is a relaxation, and it is deliberate and documented in `scaler.py`. The alternative, per-client scalers, makes each client's weights describe a different input space, so averaging them is not meaningful — the federation breaks worse. What crosses the boundary is 30 medians and 30 IQRs, not records. In deployment this is one DP-protected secure-aggregation round over feature quantiles. It is also the standard treatment in the FL-IDS literature.

"Isn't a white-box GAN attack unrealistically strong?"

Yes, intentionally. It is the strongest reasonable adversary, so the measured defence is a lower bound on robustness. A black-box attacker would do strictly worse. The budget sweep exists precisely so the result is not a single number from one arbitrary power level: it shows evasion rising $3.8\% \rightarrow 54\%$ for the undefended model across $\epsilon = 0.25 \rightarrow 1.5$, and $1.3\% \rightarrow 8.7\%$ for the defended one.

"Adversarial training usually costs clean accuracy. Yours improved. Isn't that suspicious?"

It is worth scrutiny, and I present it as an observation with a proposed mechanism rather than a law. Clean accuracy went $0.9665 \rightarrow 0.9760$ while evasion fell $28.3\% \rightarrow 4.3\%$. The likely reason is that the GAN's perturbations are bounded and mask-respecting, so they act as targeted augmentation along exactly the directions the detector was weakest in, and the model had enough spare capacity (18.7k parameters) to absorb them. A larger budget would almost certainly reverse the sign — the sweep already shows defended clean accuracy sliding from 0.9752 at $\epsilon = 0.25$ to 0.9573 at $\epsilon = 1.5$.

"How do you know your dataset isn't leaking?"

Three structural defences and one empirical test. The split happens *before* the scaler is fitted; the split is at the graph level so no test node reaches the model through message passing; sequences are built strictly inside one split. Then `verify_dataset.py` check 3 measures it: train medians must be centred near zero while test medians remain independent — identical medians would be evidence the scaler saw the test set.

"Why 4 hospitals? Why 1-second windows? Why 30 features?"

Four hospitals is the smallest number for which Krum's guarantee condition $n > 2f + 2$ is meaningful at $f = 1$, and it is configurable (`federated.n_clients`). One second is short enough that a burst attack occupies several windows (giving the model repeated evidence) and long enough for rate features to be statistically stable. Thirty

features covers seven behavioural families, of which one — `vital_zscore_max` — is MIoT-specific and does not appear in generic network IDS feature sets.

"What happens if a hospital is compromised?"

That is Step 8, and the code is complete but the experiment has not been run. Four poisoning modes are implemented (`sign_flip`, `scale`, `gauss`, `label_flip`) and five aggregation rules are ready (FedAvg, Krum, Multi-Krum, trimmed mean, coordinate median), along with the `byzantine_comparison` figure generator. I can state the expected result — FedAvg has a breakdown point of zero, so a single scaled update should be able to replace the global model, while Krum and the median should absorb it — but I have not measured it yet.

"Where is the differential privacy?"

Not built. Opacus 1.6 is installed, `PrivacyConfig` is fully specified (clipping norm, noise multiplier, target δ , target ϵ , RDP/PRV accountant choice), the client-side hook `self.privacy_engine` and the `privatise()` call site exist in `HospitalClient.fit()`, the training loop accepts a pre-built Opacus optimizer, the architecture is BatchNorm-free and uses only Opacus-compatible layers, and the `privacy_tradeoff` figure generator is written. What is missing is `swarmdef/privacy/engine.py` itself. This is a deliberate ordering choice — DP noise on top of an unmeasured baseline would be uninterpretable — not an oversight.

12. Honest status — what is not done, and known issues

READ THIS BEFORE THE REVIEW

Everything below is a real gap or a real defect. Knowing them and naming them first is far stronger than being shown them.

12.1 Not implemented

Item	What exists	What is missing
Step 7 — Differential privacy	Opacus installed; PrivacyConfig complete; client hook and privatise() call site wired; training loop accepts an external optimizer; BatchNorm-free architecture; privacy_tradeoff() plotting function ready	The swarmdef/privacy/ package is empty apart from a docstring. No DP-SGD engine, no ϵ accounting, no privacy/utility experiment.
Step 8 — Byzantine experiments	All 5 aggregators implemented and unit-usable; all 4 poisoning modes implemented; byzantine_comparison() figure ready; config fields present	No experiment script, and no run. byzantine_clients is 0 in every result on disk.
CIC-IoT2023 validation	Complete 256-line loader with a full 33-label mapping	No CSVs in data/raw/. Never executed on real data.
Tests	tests/ directory exists	Empty. No unit tests anywhere in the repo. verify_dataset.py is the only automated correctness check.
Docs	docs/ directory exists	Empty.
Step 9/10	11 figures generated; the plotting module is complete	No consolidated final evaluation sweep or written report.

12.2 Defects and inconsistencies in what is built

1. The Flower run on disk did not converge important

results/logs/step5_flower.csv shows accuracy frozen at **0.7304** and AUC at **0.403** for all 8 rounds. Identical values across every round, plus an AUC below 0.5, is the exact signature of a global model stuck at its random initialisation — the failure mode documented in flower_app.py's own docstring, where Ray workers cannot receive the client objects. The file is the most recently written log in the repository, so as things stand **the Flower deployment path is not verified working**. The custom simulator (step5_fedavg_gnn.csv) converged correctly and is the source of every federated number quoted in this document.

What to say: "The experiments run on our own deterministic simulator, which converges correctly. The Flower path demonstrates that the same clients and aggregators run under a real framework; the last recorded Flower run hit a Ray worker-serialisation issue that is documented in the code, and re-verifying it is an open item."

2. The README is out of date easy fix

The README.md still says "**Status: Steps 1–3 complete and verified. Steps 4–10 pending**" — but Steps 4, 5 and 6 are all built and have results on disk. It also quotes dataset figures from an earlier build that no longer match:

Quantity	README says	Actually on disk
attack fraction	27.5%	24.92%
train / val / test	8820 / 1260 / 2520	8824 / 1272 / 2504
split mode	row-level stratified	group-level (inductive)

The 12,600-row total and the 0.284 non-IID TV distance are still correct. This is worth fixing before the panel opens the README.

3. Duplicate Step 4 figures

`step4_architectures_f1.png` / `_acc.png` are from an earlier run and are superseded by the `_binary` / `_multiclass` versions. Harmless, but confusing if someone opens the figures folder.

4. Step 6 ran at $\epsilon = 1.0$, while the config default is $\epsilon = 0.5$

Not a defect, but know it: the headline 28.3% $\rightarrow$ 4.3% evasion figures are at $\epsilon = 1.0$, which is *twice* the configured default. At the default $\epsilon = 0.5$ the corresponding numbers are 9.0% $\rightarrow$ 1.5%, which is in the sweep table. Quote whichever you like, but say which.

5. No unit tests

Correctness rests on `verify_dataset.py`'s six checks and on the invariants asserted inside the code (budget compliance, frozen-feature checks, degenerate-shard guards). Worth saying plainly if asked about software quality.

12.3 The three-line honest summary

Built and measured: a complete MIoT digital twin, a verified non-IID federated dataset, four detector architectures, federated learning with five aggregation rules, and a full adversarial attack/defence loop with a budget sweep — Steps 1 through 6.

Built but not run: the Byzantine aggregators and poisoning modes (Step 8), and the CIC-IoT2023 real-data path.

Not built: differential privacy (Step 7), unit tests, and the final consolidated report.

13. How to reproduce every result

Setup

```
cd "Federated-Adversarial-Defenses-for-Medical-IOT"
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
brew install mosquitto          # macOS; or: sudo apt install mosquitto
```

Step 2 — the live demo (three terminals)

```
# terminal 1 - stream telemetry (starts Mosquitto automatically)
.venv/bin/python scripts/run_twin.py stream --hospitals 2 --duration 120

# terminal 2 - watch it live
.venv/bin/python scripts/run_twin.py monitor --duration 120

# terminal 3 - fire attacks on demand, mid-stream
.venv/bin/python scripts/run_twin.py attack --kind DDoS --hospital 0 --intensity 0.9
.venv/bin/python scripts/run_twin.py attack --kind MITM --hospital 0 --target infusion_pump
```

Attack kinds: DDoS, MITM, FirmwareTamper, Spoofing, Recon, Mirai.

Step 3 — build and verify the dataset

```
.venv/bin/python scripts/build_dataset.py --force
.venv/bin/python scripts/verify_dataset.py          # 6 checks, exits non-zero on failure
```

Step 4 — centralised baseline

```
.venv/bin/python scripts/train_baseline.py          # all 4 architectures, binary
.venv/bin/python scripts/train_baseline.py --multiclass # 7-way attack typing
.venv/bin/python scripts/train_baseline.py --arch gnn # just the GNN
```

Step 5 — federated learning

```
.venv/bin/python scripts/train_federated.py
.venv/bin/python scripts/train_federated.py --compare-local # + isolated-hospital baseline
.venv/bin/python scripts/train_federated_flower.py --rounds 10 # real Flower deployment
```

Step 6 — the adversarial loop

```
.venv/bin/python scripts/train_adversarial.py --rounds 12
.venv/bin/python scripts/train_adversarial.py --rounds 12 --set gan.epsilon=1.0
```

Useful overrides (any script)

```
--set federated.aggregator=krum          # fedavg | krum | multikrum | trimmed_mean | median
--set federated.byzantine_clients=1      # compromise the last N hospitals
--set federated.byzantine_attack=scale   # sign_flip | gauss | scale | label_flip
--set data.partition=dirichlet --set data.dirichlet_alpha=0.2
--set detector.arch=transformer
--set gan.epsilon=1.5
--set seed=7
```


Reproducibility guarantee

Every run is seeded through `swarmdef/utils/seed.py` and fully described by its `Config`, which is serialised alongside the results. `data/processed/dataset_meta.json` records the source, split sizes, class counts, skew statistics and seed for whatever dataset is currently on disk, so any number in this document can be traced back to the exact configuration that produced it.

End of build log · Steps 1–6 complete · 48 files · 6,720 lines · seed 42